CRUD Spring Boot - JavaScript - MySQL

Requisitosque empresa tiene 5g en argentina

IDE - Netbeans / Intellij
VSCode
XAMPP
Postman

Tecnologías de desarrollo













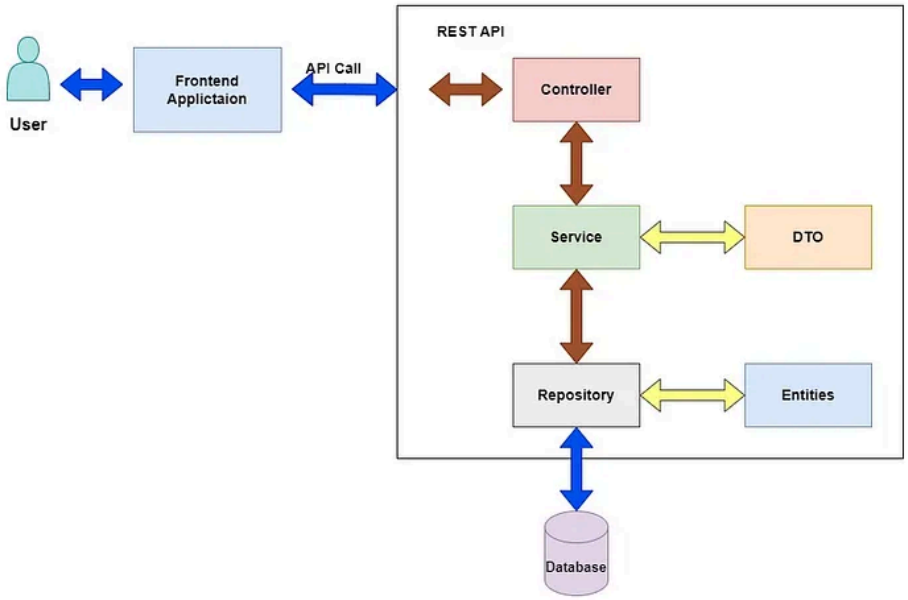
Introducción

Vamos a explicar en principio en qué consiste esta aplicación. Se trata de un simple CRUD full stack, vamos a tener un frontend y backend. Las tecnologías utilizadas del lado backend es el framework de java Spring Boot y una base de datos MySql. Por el lado del frontend es lo clásico html + js + css.

Backend:

Se trata de una API Rest construida en SB (como lo comente anteriormente), siguiendo, en lo posible, y digo lo posible para no caer en una aplicación compleja y difícil de implementar, todas las restricciones de una arquitectura REST. Quiero decir que vamos a dividirla en capas, que tendrá una interfaz uniforme, una representación JSON, etc.

Flujo del backend



Analicemos la arquitectura utilizada en el desarrollo de API REST con Spring Boot.

Funcionamiento (📣leer importante)

Pensemos que el usuario ha realizado una llamada API a través de la aplicación front-end (por ejemplo, haga clic en un botón para ver los detalles del producto) . Luego, la aplicación de front-end inicia una solicitud de API y llega a la capa de controlador de la API REST.

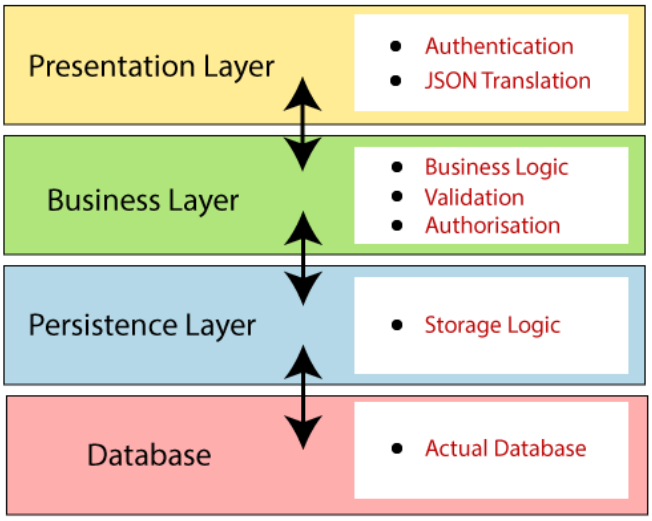
Desde la capa del **Controlador**, se reenvía la solicitud a la capa de **Servicio**. Luego, la capa de servicio reenvía la solicitud a la capa de **Repositorio**.

La capa del repositorio interactúa con la capa de la base de datos. Métodos como ***findAll***, ***findById*** y ***save*** están disponibles en la clase de repositorio. Cuando solicitamos todos los detalles del producto, la clase de servicio invoca el método findAll en la clase de repositorio.

Al recuperar o insertar datos en la base de datos mediante la capa de repositorio, se necesita un conjunto de variables para almacenar los datos. Usamos la clase **Entidad** para eso. La clase de entidad se asigna a una tabla en la base de datos. Si recuperamos los detalles del Producto de la base de datos, entonces necesitamos una clase de entidad para que el Producto encapsule los detalles de sus atributos.

A veces, el usuario del front-end no envía los datos exactos al backend que necesitamos almacenar en la base de datos. Para recuperar los datos enviados por el usuario frontend necesitamos tener una clase separada. Para este tipo de uso, utilizamos clases DTO.

Representación del flujo de datos en las capas del backend



Capa de presentación: la capa de presentación maneja las solicitudes HTTP, traduce el parámetro JSON a objeto, autentica la solicitud y la transfiere a la capa de negocio. En resumen, consta de vistas , es decir, la parte frontal.

Capa de negocio: la capa empresarial maneja toda la **lógica empresarial** . Consta de clases de servicios y utiliza servicios proporcionados por capas de acceso a datos. También realiza **autorización** y **validación** .

Capa de persistencia: la capa de persistencia contiene toda la **lógica de almacenamiento** y traduce los objetos de negocio desde y hacia las filas de la base de datos.

Capa de base de datos: en la capa de base de datos se realizan operaciones **CRUD (crear, recuperar, actualizar, eliminar).**

COMENCEMOS CON EL CÓDIGO!!!!✍️

Vamos a dividir toda la práctica por TAREAS 🗒️

TAREA 1

- Crear la base de datos.
- Crear la estructura de carpetas

TAREA 2

- Crear el proyecto en la página de https://start.spring.io/
- Agregar las dependencias necesaria

TAREA 3

- Verificar que el proyecto despliegue. Aca solo miramos que todo funcione antes de empezar a codificar para no arrastrar errores en la descargas de las dependencias de maven

TAREA 4

- Dividir el proyecto en las siguientes capas:
    - **contreller**
    - **service**
    - **entity**
    - **repository**

TAREA 5

- Configurar el archivo ***application.properties*** para acceder a la base de datos. En este archivo se configura la url y el driver para acceder a la db creada en el paso 1

TAREA 6

- En la capa Entity crear la clase que encapsula los datos que van a interactuar con la base de datos. Para este proyecto creamos la clase **Product.**
- Realizar las anotaciones de JPA para mapear con la tabla en db.

TAREA 7

1. En la capa Repository crear la interfaz **ProductRepository** y heredar de CRUDJPARepository

TAREA 8

- En la capa service crear la clase **ProductService** e implementar los métodos **CRUD** definidos en el framework JPA usando inyección de dependencias

TAREA 9

- En la capa controller crear la clase **ProductController**
- Inyectar la dependencia de la capa service
- Crear los **endpoint** con las respuestas al cliente.

TAREA 10

- Probar los endpoint con **Postman**

Vamos a empezar a caminar 🚶

TAREA 1

1. **Crear la base de datos**

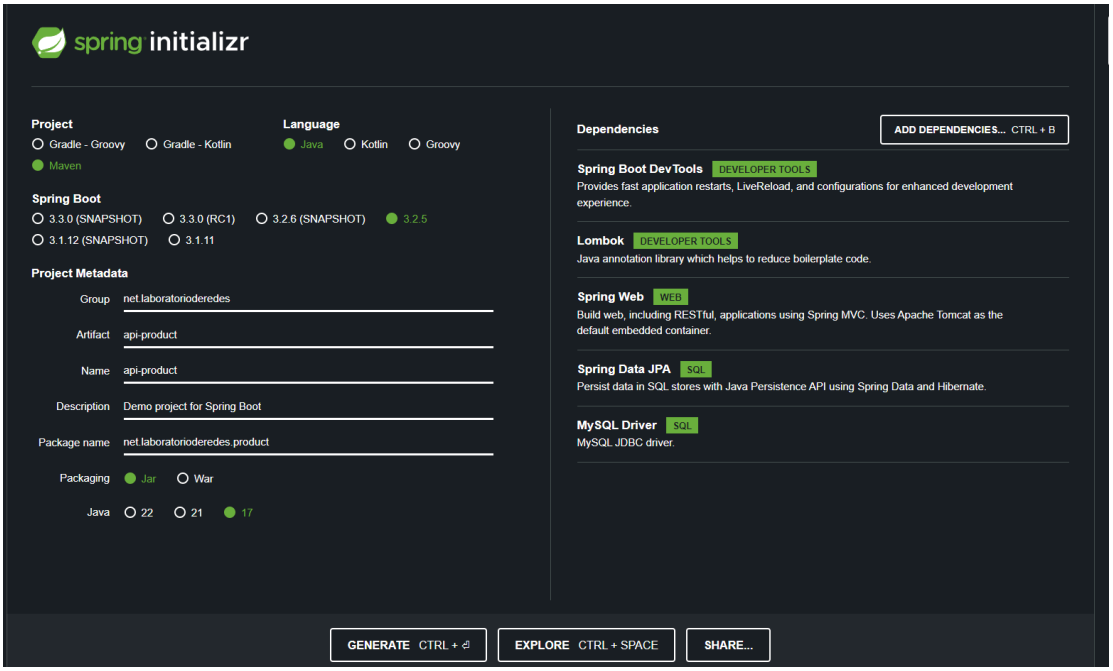
```
CREATE DATABASE db_product;
```

2. **Crear la siguiente estructura de carpetas**

```
C:\USERS\VICEC\DESKTOP\FULL-STACK
├────back
└────front
```

TAREA 2

1. **Generar un proyecto en https://start.spring.io/ descargarlo y descomprimir en la carpeta back. Agregar las dependencias que se describen en la imagen. Abrir el proyecto con tu IDE de preferido, en mi caso voy a usar Intellij**

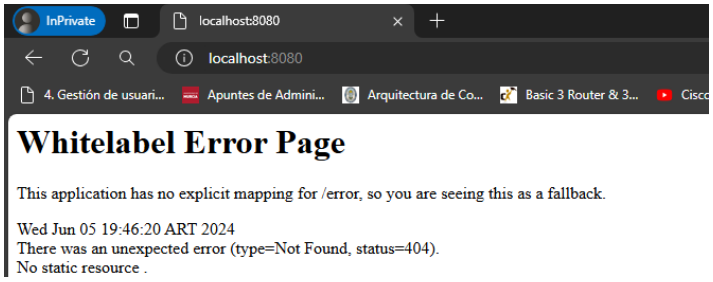


TAREA 3

1. **Verificar despliegue**
   Como aún no tenemos configurada la conexión a la base de datos, es necesario comentar las dependencias de JPA en el archivo pom.xml

```
<!--        <dependency>-->
<!--            <groupId>org.springframework.boot</groupId>-->
<!--            <artifactId>spring-boot-starter-data-jpa</artifactId>-->
<!--        </dependency>-->
```

   Una vez comentada la dependencia iniciamos la aplicación, verificamos en http://localhost:8080

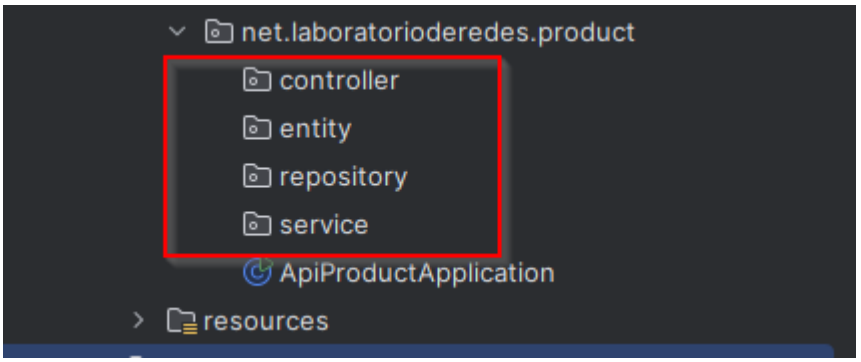


También podemos ver los logs en el IDE

```
erver  : Tomcat started on port 8080 (http) with context path ''
       : Started ApiProductApplication in 3.548 seconds (process running for 4.074)
       : Initializing Spring DispatcherServlet 'dispatcherServlet'
       : Initializing Servlet 'dispatcherServlet'
       : Completed initialization in 1 ms
```

TAREA 4

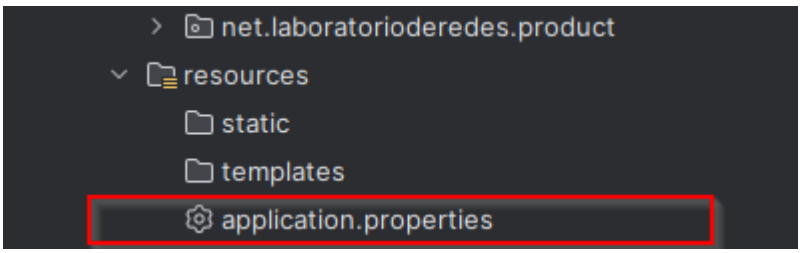
En la siguiente captura de pantalla muestro los paquetes que cree



Con esta estructura de paquetes cumplimos con una de las restricciones REST. En ocasiones es posible que veamos aplicaciones con más capas(paquetes), en este caso por cuestiones de simplicidad nos limitaremos a las capas esenciales.

TAREA 5

1. Configurar elarchivo ***application.properties.***



Hay una gran cantidad de propiedades que se pueden establecer, podemos ver una lista aquí.
Observa, que existen dieciséis categorías que clasifican a las propiedades, que van desde el 'core' de Sprint, pasando por configuraciones de datos hasta testing.
En este archivo application.properties estamos usando las propiedades ya definidas por Spring para darle un nombre a nuestra aplicación e indicarle configuraciones sobre base de datos.

```
spring.application.name=api-product
#DATABASE
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost:3306/db_product
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.show-sql= true
```

TAREA 6

1. En la capa (paquete) entity creamos la clase **Produc** con las anotaciones de JPA y de lombok
2. También tenemos que descomentar las dependencias en el archivo pom.xml para utilizar las anotaciones de JPA

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
</dependency>
```

```
kage net.laboratorioderedes.product.entity;

ort jakarta.persistence.GeneratedValue;
ort jakarta.persistence.GenerationType;
ort jakarta.persistence.Id;
ort jakarta.persistence.Table;
ort lombok.AllArgsConstructor;
ort lombok.Data;
ort lombok.NoArgsConstructor;

ArgsConstructor // Lombok annotation to generate a no-args constructor
lArgsConstructor // Lombok annotation to generate an all-args constructor
ta //Lombok annotation to generate setter and getter
tity //JPA annotation to specify entity
ble(name = "products") // JPA annotation to specify the table name for this
ity
lic class Product {
//---------------------------------
@Id //JPA annotation to specify primary key
@GeneratedValue(strategy = GenerationType.IDENTITY) //JPA annotation to specify
o-increment
private Long id; //this attribute is primary key and auto_increment
//-----------------------------------
private String name;
private double price;
private int quantity;
```

TAREA 7

1. En la capa Repository crear la interfaz **ProductRepository** y heredar de CRUDRepository y anotar la interfaz con **`@Repository`**

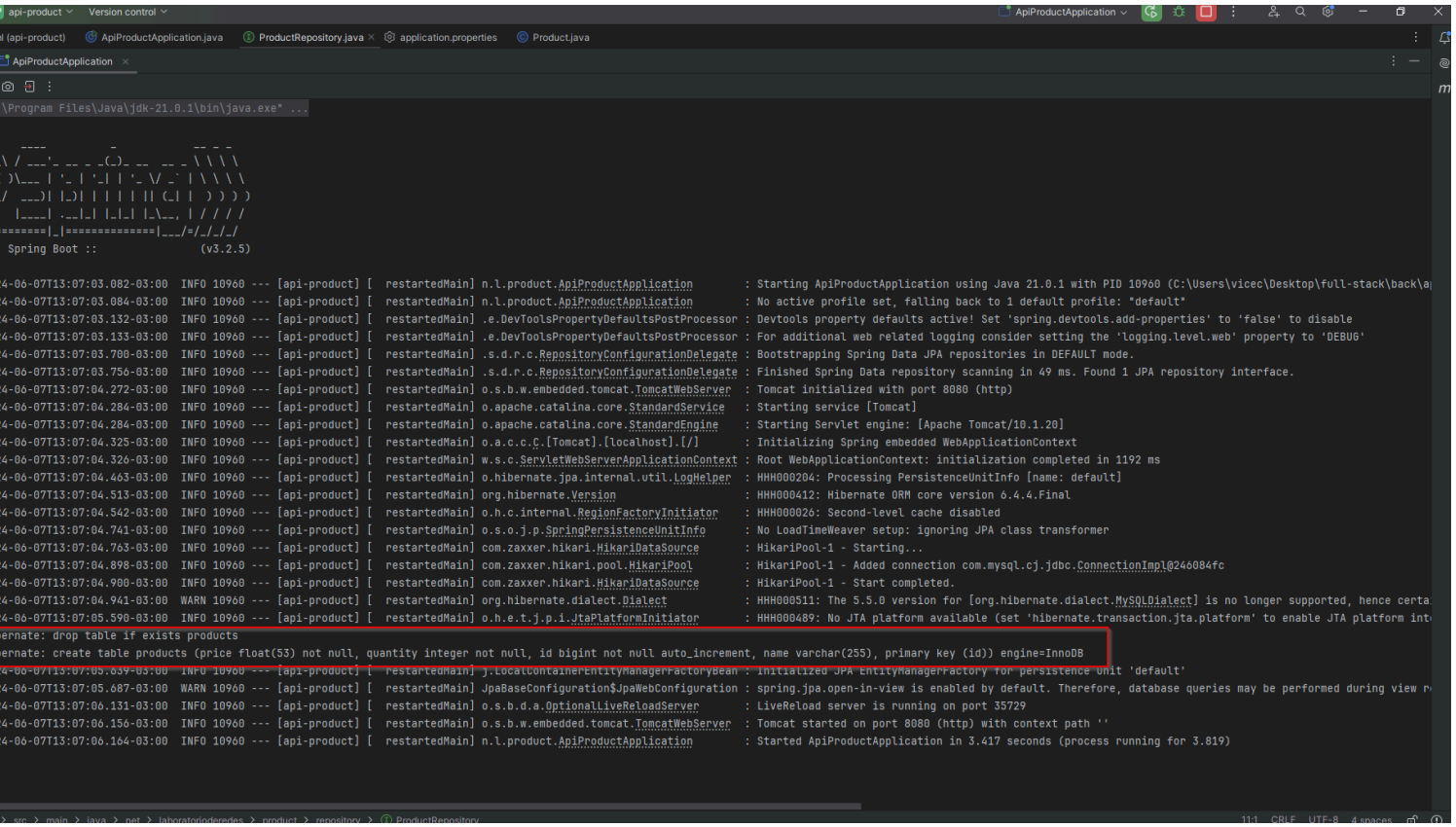
```
package net.laboratorioderedes.product.repository;

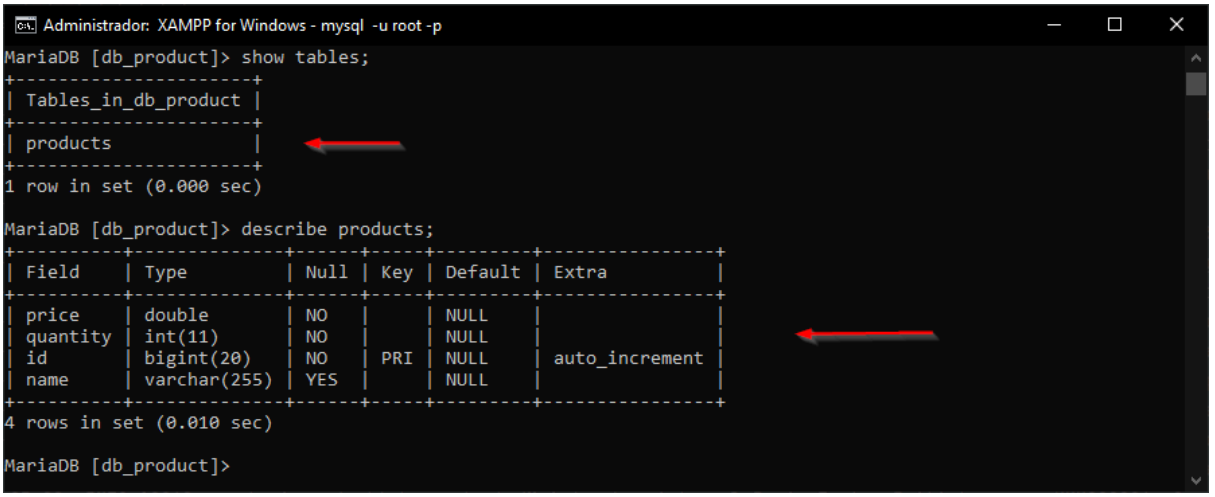

import net.laboratorioderedes.product.entity.Product;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product,Long> {
}
```

2. Probar funcionalidad. Con esta prueba esperamos que nos crea la tabla ***products*** con todos los campos establecidos en la clase Product
   Importante!!! no olvidar tener encendido el servidor de base de datos

3. Vemos los resultados en los logs y en el servidor de db





Funcionando 👌

TAREA 8

- En la capa service crear la clase **ProductService** e implementar los métodos **CRUD** definidos en el framework JPA usando inyección de dependencias

Para cumplir con esta tarea tenemos que crear en la capa service, una interface con la firma de los métodos y una clase que implemente los métodos definidos en la interface y anotarla con **@Service**

//código de la interface

```
package net.laboratorioderedes.product.service;
```

```java
import net.laboratorioderedes.product.entity.Product;

import java.util.List;

public interface ProductService {
    //create
    public Product createProduct(Product product);

    //read
    public List<Product> getAllProducts();
    public Product getProductById(Long productId);

    //update
    public Product updateProduct(Long productId, Product product);

    //delete
    public Product deleteProductById(Long productId);

}
```

//código de la clase.

Observar la annotation @Autowired que se utiliza para inyectar la dependencia del repositorio, con esta anotación evitamos utilizar el operador new, y dejamos que el framework de spring instancie el objeto por nosotros y lo inyecte cuando lo necesitamos

```java
age net.laboratorioderedes.product.service;

rt jakarta.persistence.EntityNotFoundException;
rt net.laboratorioderedes.product.entity.Product;
rt net.laboratorioderedes.product.repository.ProductRepository;
rt org.springframework.beans.factory.annotation.Autowired;
rt org.springframework.stereotype.Service;

rt java.util.List;
rt java.util.Optional;

vice
ic class ProducServiceImpl implements ProductService {

Autowired
rivate ProductRepository productRepository;

Override
ublic Product createProduct(Product product) {
   return productRepository.save(product);


Override
ublic List<Product> getAllProducts() {
```

```
    return productRepository.findAll();


Override
ublic Product getProductById(Long productId) {
    return productRepository.findById(productId)
            .orElseThrow(() -> new EntityNotFoundException("Product not found with id " +
uctId));


Override
ublic Product updateProduct(Long productId, Product product) {
        Optional<Product> optionalProduct = productRepository.findById(productId);
        if (optionalProduct.isPresent()) {
            Product existingProduct = optionalProduct.get();
            existingProduct.setName(product.getName());
            existingProduct.setPrice(product.getPrice());
            existingProduct.setQuantity(product.getQuantity());
            return productRepository.save(existingProduct);
        }
        return null;
    Product existingProduct = productRepository.findById(productId)
            .orElseThrow(() -> new EntityNotFoundException("Product not found with id " +
uctId));
    //update field
    existingProduct.setName(product.getName());
    existingProduct.setPrice(product.getPrice());
    existingProduct.setQuantity(product.getQuantity());
    System.out.println(product);
    return productRepository.save(existingProduct);



Override
ublic Product deleteProductById(Long productId) {
        Optional<Product> optionalProduct = productRepository.findById(productId);
        if (optionalProduct.isPresent()) {
            productRepository.deleteById(productId);
            return optionalProduct.get();
        }
        return null;
    }
    return productRepository.findById(productId)
            .orElseThrow(() -> new EntityNotFoundException("Product not found with id " +
uctId));
```

TAREA 9

- En la capa controller crear la clase **ProductController**
- Inyectar la dependencia de la capa service
- Crear los **endpoint** con las respuestas al cliente.

```
ge net.laboratorioderedes.product.controller;

t jakarta.persistence.EntityNotFoundException;
t net.laboratorioderedes.product.entity.Product;
t net.laboratorioderedes.product.service.ProductService;
t org.springframework.beans.factory.annotation.Autowired;
t org.springframework.http.HttpStatus;
t org.springframework.http.ResponseEntity;
t org.springframework.web.bind.annotation.*;

t java.util.List;

Controller
estMapping("/api/v1")
c class ProductComtroller {

inject dependency
utowired
ivate ProductService productService;

ostMapping("/products")
method to save product
blic ResponseEntity<Product> saveProduct(@RequestBody Product product) {
   System.out.println(product);
   return
nseEntity.status(HttpStatus.CREATED).body(productService.createProduct(product));


etMapping("/products")
method to get all products
blic ResponseEntity<List<Product>> getAllProducts() {
   return ResponseEntity.ok(productService.getAllProducts());


etMapping("/products/{id}")
method to get product by id
blic ResponseEntity<Product> getProductById(@PathVariable Long id) {
   return ResponseEntity.ok(productService.getProductById(id));


utMapping("/products/{id}")
method to update product
blic ResponseEntity<Product> updateProduct(@PathVariable Long id, @RequestBody
ct product) {
```

```
    System.out.println(id);
    System.out.println(product);
    Product existingProduct = productService.getProductById(id);
    if (existingProduct == null) {
        throw new EntityNotFoundException("Product not found with id: " + id);
    }
    return ResponseEntity.ok(productService.updateProduct(id, product));

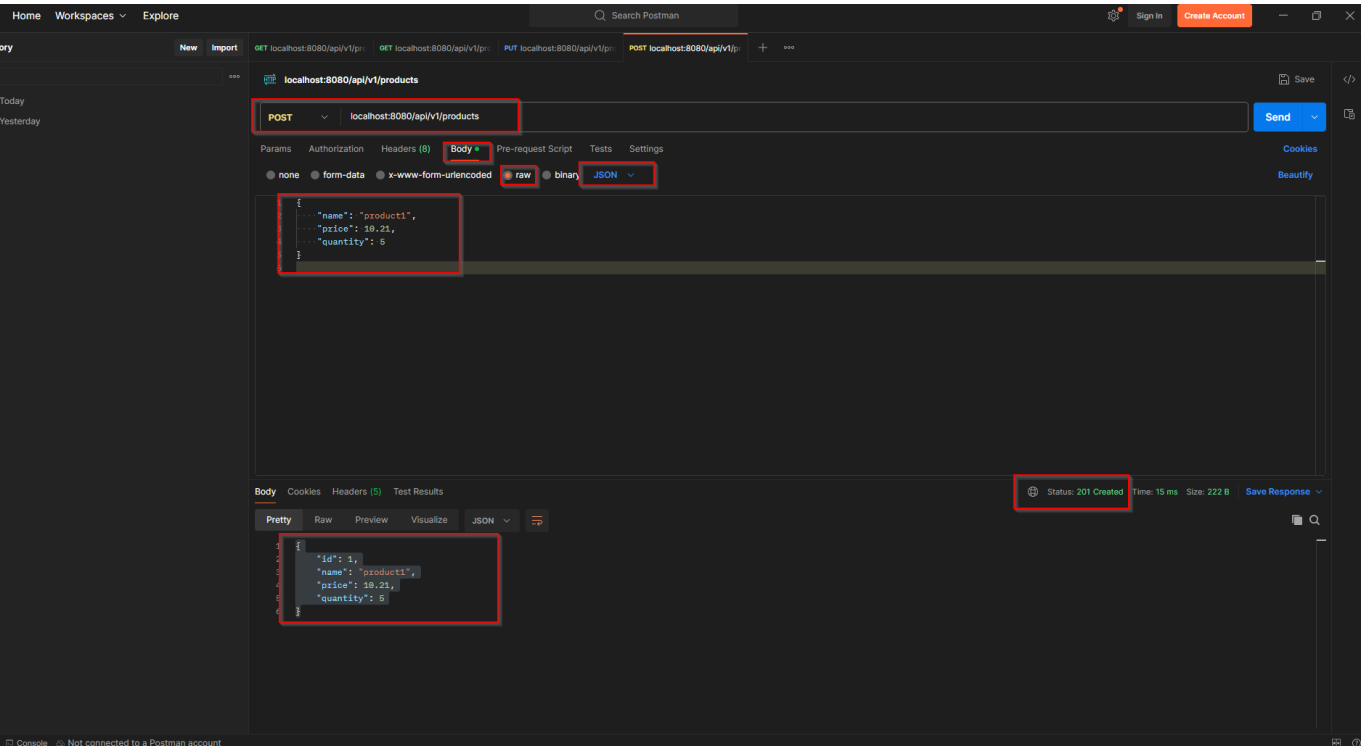

eleteMapping("/products/{id}")
/method to delete product
blic ResponseEntity<String> deleteProduct(@PathVariable Long id) {
    productService.deleteProductById(id);
    return ResponseEntity.ok("Product deleted successfully");


xceptionHandler(EntityNotFoundException.class)
blic ResponseEntity<String> handleEntityNotFoundException(EntityNotFoundException
{
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
```

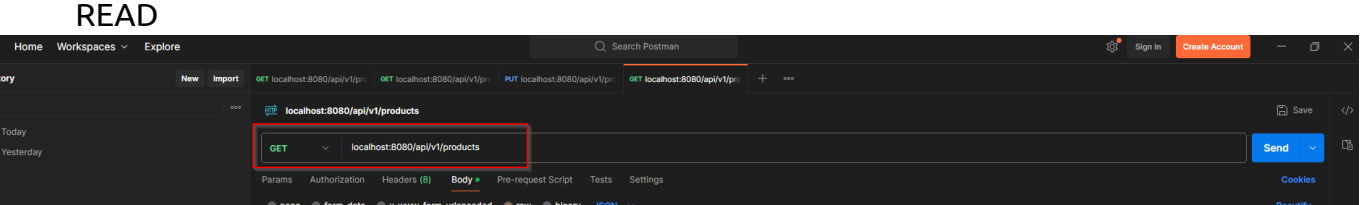
TAREA 10

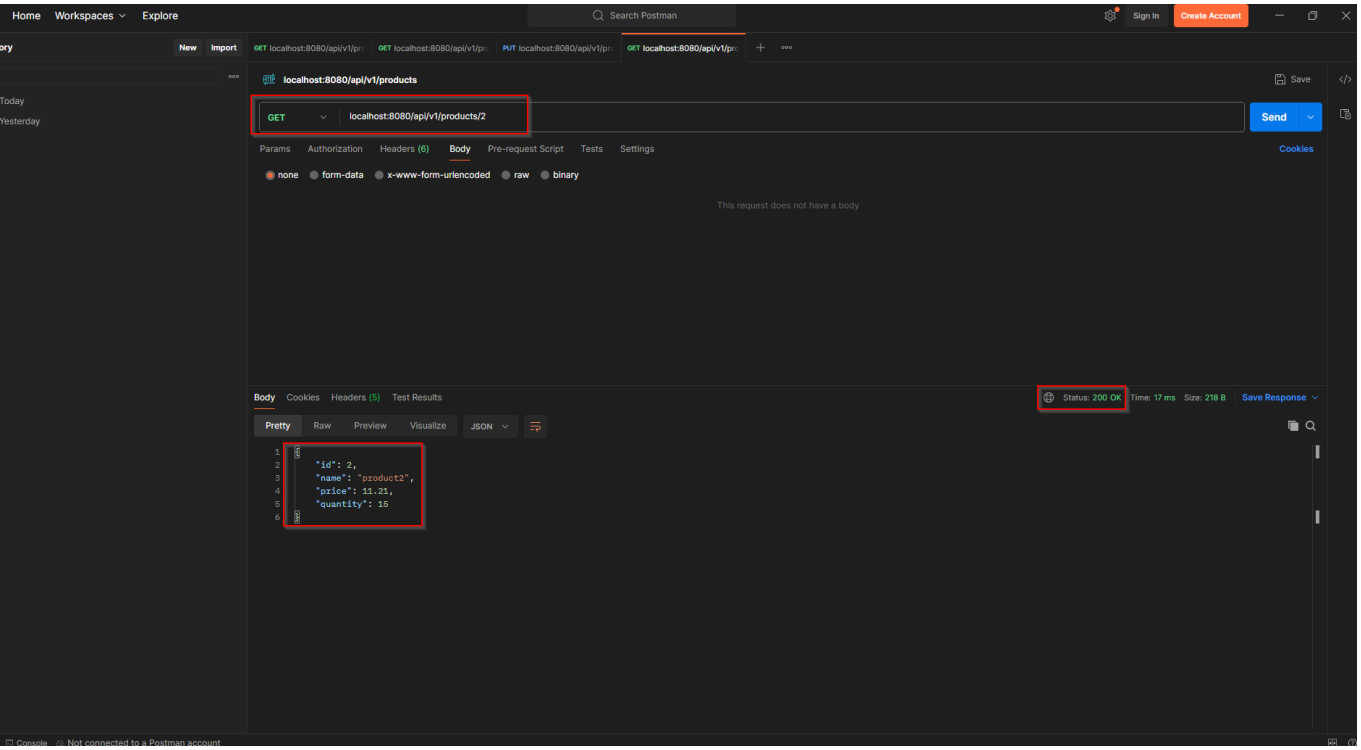
- Probar los endpoint con **Postman**

CREATE

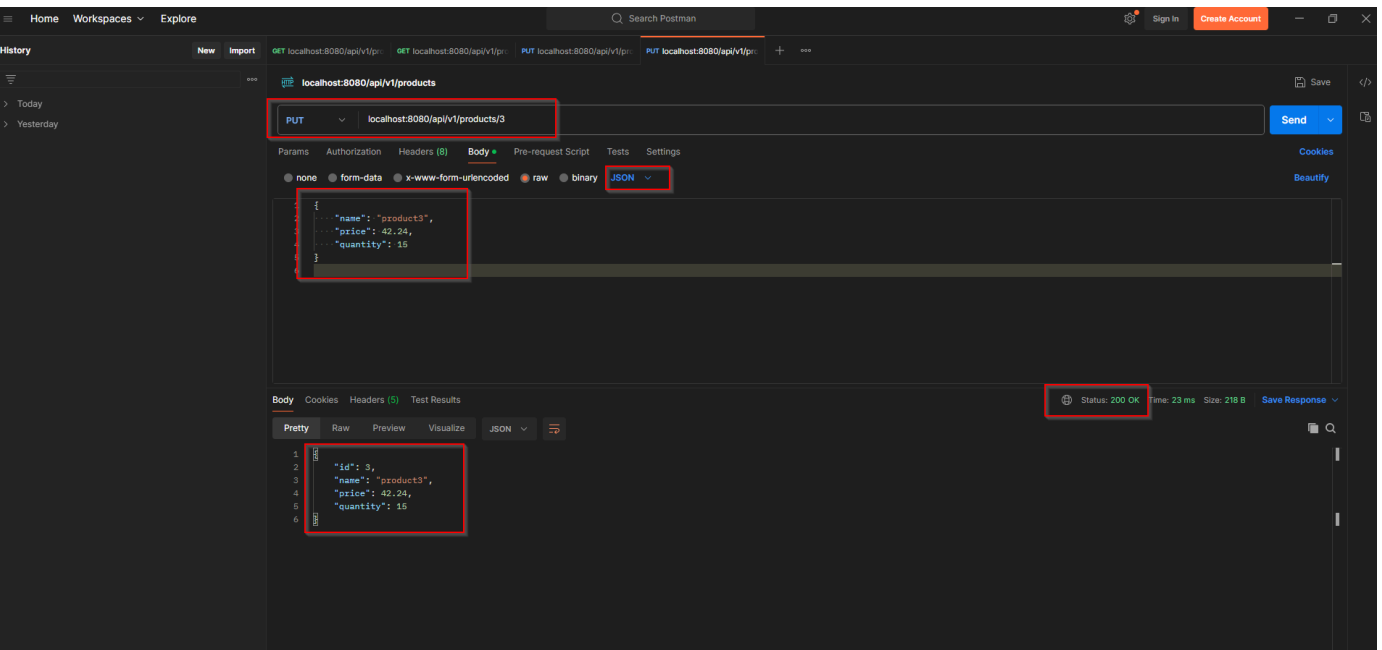


READ

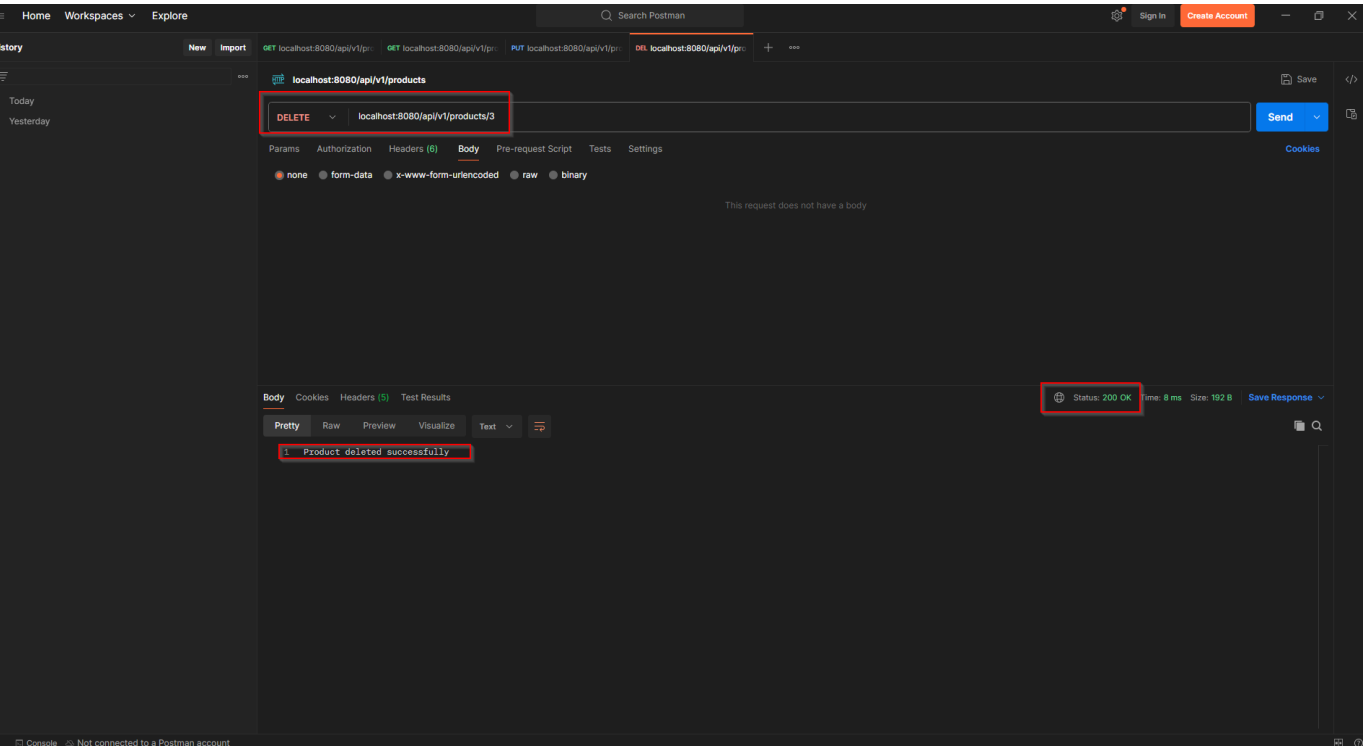



Home
Workspaces
Explore
Search Postman
Sign In
Create Account
localhost:8080/api/v1/products
GET
localhost:8080/api/v1/products/2
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
This request does not have a body
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 17 ms
Size: 218 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"id": 2,
"name": "product2",
"price": 11.21,
"quantity": 15
}

UPDATE


Home
Workspaces
Explore
Search Postman
Sign In
Create Account
localhost:8080/api/v1/products
PUT
localhost:8080/api/v1/products/3
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"name": "product3",
"price": 42.24,
"quantity": 15
}
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 23 ms
Size: 218 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"id": 3,
"name": "product3",
"price": 42.24,
"quantity": 15
}

DELETE



Frontend:

Frontend es la parte de la aplicación al que el usuario puede acceder directamente. En el contexto de diseño web y desarrollo web, se refiere a todas las tecnologías que corren en el navegador y que se encargan de la interactividad con los usuarios.

Dicho esto las tecnologías que vamos a utilizar son las siguientes:

- **VSCode (como editor de código)**
- **HTML**
- **Javascript**
- **CSS(opcional)**
- **Bootstrap**

Funcionamiento (📣leer importante)

Antes de comenzar a codificar, es mejor entender previamente el flujo de datos en el frontend. En términos generales, el flujo de datos sigue estos pasos:

1. **Cuando se carga el DOM se produce el evento `DOMContentLoaded`, ahí es donde vamos a ejecutar la función principal que es la encargada de obtener todos los datos del servidor.**

2. Una vez que obtenemos los datos, llamamos a una función para renderizar la lista de productos en el DOM.
3. Junto con la renderización de los datos en el DOM creamos una tabla con los botones para interactuar con la aplicación mediante los métodos `UPDATE` y `DELETE`
4. En el HTML creamos un formulario para manejar las solicitudes `POST`, `PUT`

Al igual que hicimos en el backend, vamos a dividir la aplicación en varias tareas

Tarea 1: Configurar el entorno de desarrollo

1. **Crear un proyecto**: En la carpeta **front** crea un archivo `index.html` y otro `script.js`.
2. **Estructura básica del HTML**: En el `index.html`, vincula el archivo `script.js`. Crea un formulario con los campos que tenes en las base de datos

Tarea 2: Configurar la función para obtener datos
Crear función: En `script.js`. Define una función asíncrona para realizar una petición `GET` al backend para obtener la lista de productos.

Tarea 3: Renderizar los datos en el DOM

Crear función para renderizar: Define una función para mostrar los productos en la página web.

Tarea 4: Agregar un producto

Crea una función asíncrona para enviar por el método POST un producto al backend

Tarea 5: Actualizar un producto

Crea una función asíncrona para enviar por el método PUT un producto al backend

Tarea 6: Eliminar un producto

Crea una función asíncrona para enviar por el método DELETE una solicitud al backend para eliminar un producto.

Tarea 7: Manejar el envío de formulario

Crea una función para manejar el envío del formulario. Esta función debe llamar a los métodos guardar o actualizar según el criterio lógico que se adopte

Tarea 8: Manejar el clic en el botón de editar

Crea una función asíncrona para enviar por el método GET una solicitud al backend para obtener un producto por id.

Tarea 9: Manejar el clic en el botón de eliminar

Crear una función para obtener el id del producto y enviarlo a la función que elimina el producto del backend

Tarea 1

Configurar el entorno de desarrollo

En la carpeta `front` crear los archivos `index.html`, `script.js` y `style.css`(opcional)

Abrir el proyecto con vscode

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CRUD Productos</title>
    <link
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
    <div class="container mt-5">
        <h1 class="text-center">CRUD de Productos</h1>

        <!-- Formulario para agregar/editar productos -->
        <div class="card mb-4">
            <div class="card-body">
                <h5 class="card-title">Agregar/Editar Producto</h5>
                <form id="product-form">
                    <input type="hidden" id="product-id">
                    <div class="form-group">
                        <label for="name">Nombre</label>
                        <input type="text" class="form-control" id="name"
required>
                    </div>
                    <div class="form-group">
                        <label for="price">Precio</label>
                        <input type="number" class="form-control" id="price"
required>
                    </div>
                    <div class="form-group">
                        <label for="quantity">Cantidad</label>
                        <input type="number" class="form-control" id="quantity"
required>
                    </div>
                    <button type="submit" class="btn
btn-primary">Guardar</button>
                </form>
            </div>
        </div>

        <!-- Tabla para mostrar productos -->
```

```
        <div class="card">
            <div class="card-body">
                <h5 class="card-title">Lista de Productos</h5>
                <table class="table">
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Nombre</th>
                            <th>Precio</th>
                            <th>Cantidad</th>
                            <th>Acciones</th>
                        </tr>
                    </thead>
                    <tbody id="product-table">
                        <!-- Aquí se agregarán los productos -->
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Tarea 2 y 3:

Configurar la función para obtener datos

```
// Función para obtener todos los productos
    function fetchProducts() {
        fetch(apiUrl)
            .then(response => response.json())
            .then(data => {
                const productTable = document.getElementById('product-table');
                productTable.innerHTML = '';
                data.forEach(product => {
                    const row = document.createElement('tr');
                    row.innerHTML = `
                        <td>${product.id}</td>
                        <td>${product.name}</td>
                        <td>${product.price}</td>
                        <td>${product.quantity}</td>
                        <td>
                            <button class="btn btn-warning btn-sm edit-btn"
data-id="${product.id}">Editar</button>
                            <button class="btn btn-danger btn-sm delete-btn"
data-id="${product.id}">Eliminar</button>
```

```
                    </td>
                `;
                productTable.appendChild(row);
            });
        });
    }
```

Tarea 4: Agregar un producto

```
// Función para agregar un producto
    function addProduct(product) {
        fetch(apiUrl, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(product)
        }).then(() => fetchProducts());
    }
```

Tarea 5: Actualizar un producto

```
    // Función para actualizar un producto
    function updateProduct(id, product) {
        fetch(`${apiUrl}/${id}`, {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(product)
        }).then(() => fetchProducts());
    }
```

Tarea 6: Eliminar un producto

```
    // Función para eliminar un producto
    function deleteProduct(id) {
        fetch(`${apiUrl}/${id}`, {
            method: 'DELETE'
        }).then(() => fetchProducts());

    }
```

Tarea 7: Manejar el envío de formulario

```
 // Manejar el envío del formulario
    document.getElementById('product-form').addEventListener('submit', function(event)
{
        event.preventDefault();
        const id = document.getElementById('product-id').value;
        const product = {
            name: document.getElementById('name').value,
            price: document.getElementById('price').value,
            quantity: document.getElementById('quantity').value
        };

        if (id) {
            updateProduct(id, product);
        } else {
            addProduct(product);
        }

        document.getElementById('product-form').reset();
        document.getElementById('product-id').value = '';
    });
```

Tarea 8: Manejar el clic en el botón de editar

```
  // Manejar el clic en el botón de editar
    document.getElementById('product-table').addEventListener('click', function(event)
{
        if (event.target.classList.contains('edit-btn')) {
            const id = event.target.dataset.id;
            fetch(`${apiUrl}/${id}`)
                .then(response => response.json())
                .then(product => {
                    document.getElementById('product-id').value = product.id;
                    document.getElementById('name').value = product.name;
                    document.getElementById('price').value = product.price;
                    document.getElementById('quantity').value = product.quantity;
                });
        }
```

Tarea 9: Manejar el clic en el botón de eliminar

```
   // Manejar el clic en el botón de eliminar
        if (event.target.classList.contains('delete-btn')) {
            const id = event.target.dataset.id;
            deleteProduct(id);
        }
    });
```

Código en el repositorio de github

https://github.com/cersosimovicente/implementacion-de-proyectos/tree/main/full-stack